MECHATRONICS AND IOT

ASSIGNMENT 2

AIR POLLUTION MONITORING SYSTEM USING MQ135 AND ESP8266 WITH ADAFRUIT IO CLOUD

TEAM MEMBERS:

JEEVA T	**24MER020**
NITHEESH M	**24MER036**
SANJAY S	**24MER054**
SIVA BALAJI S	**24MER058**
SIVA S	**24MER059**

AIR POLLUTION MONITORING SYSTEM USING MQ135 AND ESP8266 WITH ADAFRUIT IO CLOUD

1. Project Overview

The Air Pollution Monitoring System is an IoT-based system designed to detect harmful gases and monitor air-quality conditions in real time. The system uses an **MQ135 gas sensor** to sense the presence of harmful gases and an **ESP8266 Node MCU** to process the sensor data.

The measured gas value is transmitted through Wi-Fi to the **Adafruit IO cloud platform**, where the data can be remotely monitored and stored for analysis. Two LEDs are used as local indicators of air quality. The green LED indicates good air quality, while the red LED indicates poor or polluted air quality.

The system can be used for basic air-quality monitoring in industrial areas, laboratories, workshops, classrooms and environmental monitoring applications.

Main components

- MQ135 gas sensor
- ESP8266 Node MCU
- Green LED
- Red LED
- 220 Ω resistors
- Wi-Fi network
- Adafruit IO cloud platform

2. Project Statement

Air pollution caused by harmful gases is a major concern in industrial and environmental applications. Continuous monitoring of air quality is necessary to identify polluted conditions and provide timely warnings.

Conventional gas monitoring systems may require manual observation and may not provide remote access to sensor data.

Therefore, an **IoT-based Air Pollution Monitoring System** is proposed to continuously measure gas levels using an MQ135 sensor, process the data using an ESP8266 Node MCU, provide visual indication using LEDs, and upload the sensor readings to a cloud platform for remote monitoring and analysis.

3. Proposed Solution

The proposed system consists of an MQ135 gas sensor connected to an ESP8266 Node MCU.

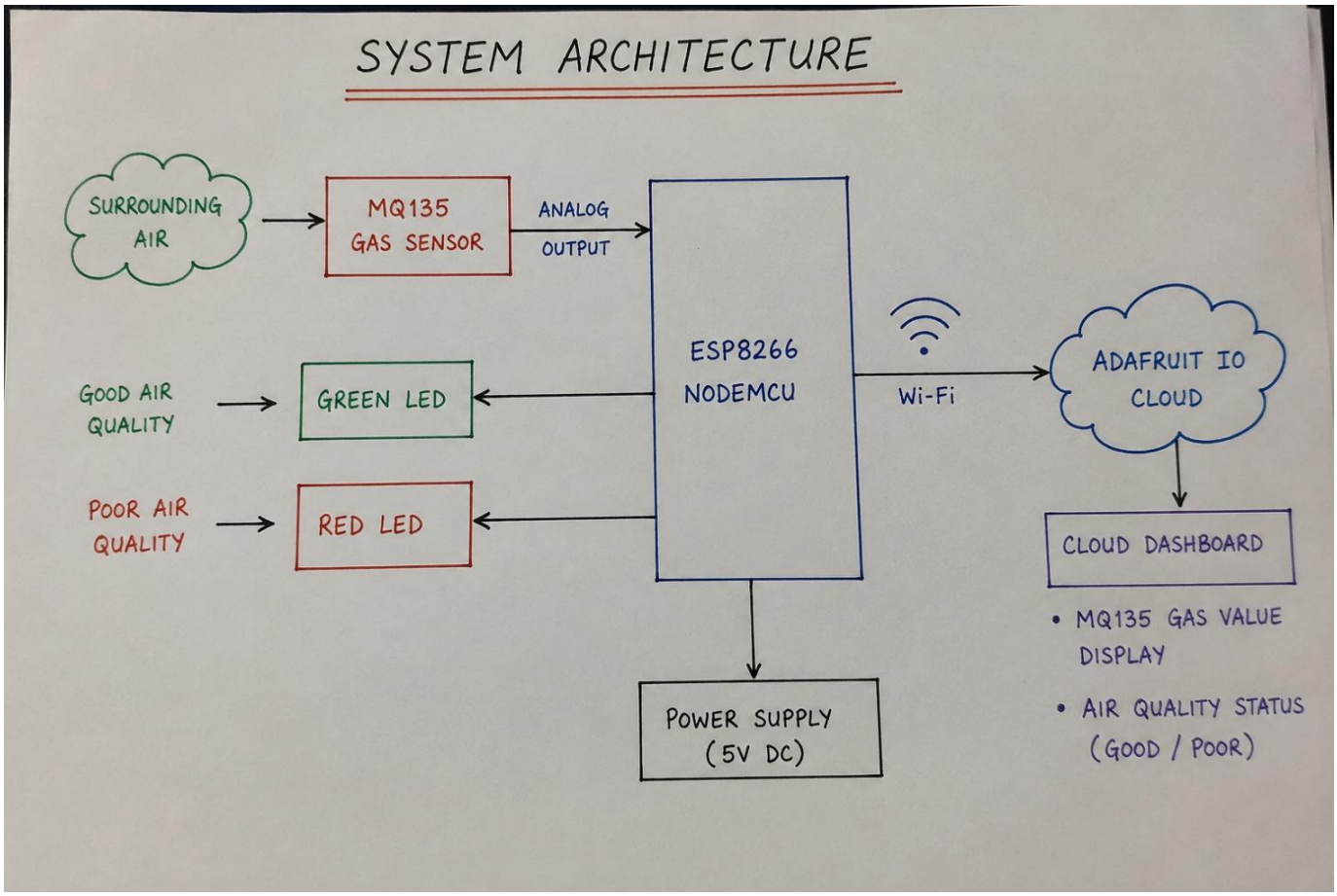
The MQ135 senses gases present in the surrounding air and produces an Analog output. The ESP8266 reads this analog value and compares it with a predefined threshold.

The system operates as follows:

1. MQ135 detects gases in the surrounding environment.
2. ESP8266 reads the sensor value.
3. The sensor value is compared with the threshold.
4. If the value is below the threshold, the air is considered **GOOD**.
5. Green LED is switched ON.

6. If the value exceeds the threshold, the air is considered **POOR/POLLUTED**.
7. Red LED is switched ON.
8. The MQ135 value and air-quality status are transmitted to **Adafruit IO** using Wi-Fi.
9. The data can be monitored remotely through the Adafruit IO dashboard.

4. System Architecture:



5. Circuit Table:

CIRCUIT TABLE

1. MQ135 GAS SENSOR CONNECTION

Component	Pin	ESP8266 (NodeMCU) Pin	Function
MQ135 Gas Sensor	VCC	VIN / 5V	Power Supply (5V)
	GND	GND	Ground
	AO	A0	Analog Output to ESP8266

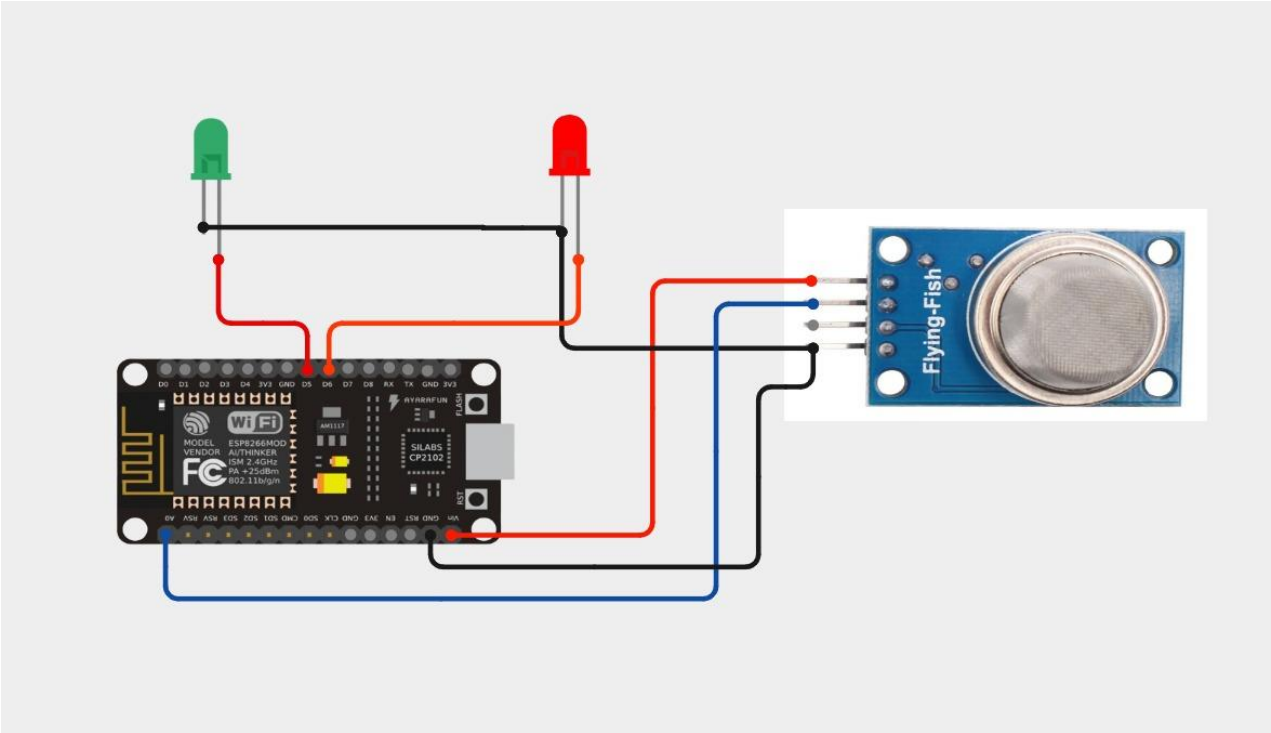
2. LED CONNECTIONS

Component	LED Pin	ESP8266 (NodeMCU) Pin	Other Connection	Function
Green LED	Anode (+)	D5 (GPIO14)	Through 220 Ω Resistor	Indicates Good Air Quality
	Cathode (–)	GND	Direct to GND	–
Red LED	Anode (+)	D6 (GPIO12)	Through 220 Ω Resistor	Indicates Poor Air Quality
	Cathode (–)	GND	Direct to GND	–

3. POWER SUPPLY

Component	Pin	Connection	Function
ESP8266 NodeMCU	VIN	5V DC (USB / External 5V Supply)	Power Supply to ESP8266
ESP8266 NodeMCU	GND	Common Ground	Ground

6. Circuit Design:



7. Algorithm:

Algorithm for Air Pollution Monitoring System

Step 1: Start the system.

Step 2: Initialize ESP8266, MQ135 sensor and LEDs.

Step 3: Connect ESP8266 to the Wi-Fi network.

Step 4: Establish connection with Adafruit IO.

Step 5: Allow the MQ135 sensor to warm up.

Step 6: Read the Analog value from MQ135.

Step 7: Compare the sensor value with the predefined threshold.

Step 8: If the value is below the threshold:

- Turn ON green LED.
- Turn OFF red LED.
- Set air quality as GOOD.

Step 9: If the value exceeds the threshold:

- Turn OFF green LED.
- Turn ON red LED.
- Set air quality as POOR/POLLUTED.

Step 10: Send MQ135 value to the Adafruit IO mq135 feed.

Step 11: Send air-quality status to the air-quality feed.

Step 12: Repeat the process continuously.

8. Coding:

```
#include <ESP8266WiFi.h>
#include "Adafruit_MQTT.h"
#include "Adafruit_MQTT_Client.h"
// ================================
// WIFI
// ================================
#define WIFI_SSID       "OnePlus Nord CE5 7C5C"
#define WIFI_PASSWORD   "qkts5737"
// ================================
// ADAFRUIT IO
// ================================
#define AIO_SERVER      "io.adafruit.com"
#define AIO_SERVERPORT 1883
#define AIO_USERNAME    "Sivasaravanan"
#define AIO_KEY         "aio_ADVS50c7oG5Sx1R3FskFAq0fjUrB"
// ================================
// PINS
// ================================
#define MQ135_PIN A0
#define GREEN_LED D5
#define RED_LED D6
// ================================
// MQTT
// ================================
WiFiClient client;
Adafruit_MQTT_Client mqtt(
  &client,
  AIO_SERVER,
```

```
  AIO_SERVERPORT,
  AIO_USERNAME,
  AIO_KEY
);
// ================================
// FEEDS
// ================================
Adafruit_MQTT_Publish mq135Feed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/mq135"
  );
Adafruit_MQTT_Publish airQualityFeed =
  Adafruit_MQTT_Publish(
    &mqtt,
    AIO_USERNAME "/feeds/air-quality"
  );
// ================================
// SETUP
// ================================
void setup()
{
  Serial.begin(9600);
  delay(1000);
  pinMode(GREEN_LED, OUTPUT);
  pinMode(RED_LED, OUTPUT);
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, LOW);
Serial.println();
```

```
  Serial.println("==============================");
  Serial.println(" AIR POLLUTION MONITORING");
  Serial.println("==============================")
  // ------------------------------
  // CONNECT WIFI
  // ------------------------------
  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }
  Serial.println();
  Serial.println("WiFi CONNECTED!");
  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());
  // ------------------------------
  // SENSOR WARM UP
  // ------------------------------
  Serial.println();
  Serial.println("MQ135 sensor warming up...");
  delay(10000);
  Serial.println("Sensor ready!");
}
// ==============================
// MQTT CONNECTION
// ==============================
void MQTT_connect()
```

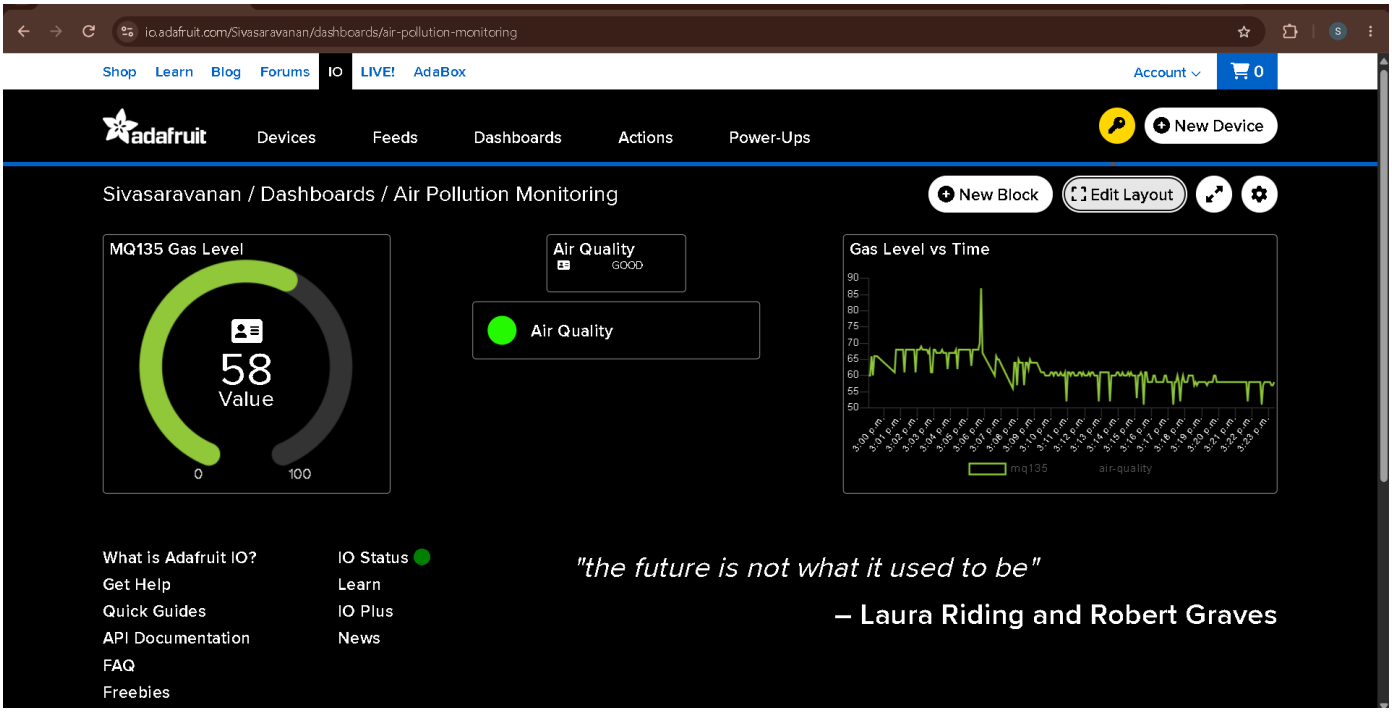
```
{
  if (mqtt.connected())
  {
    return;
  }
  Serial.println();
  Serial.print("Connecting to Adafruit IO...");
  int8_t ret;
  while ((ret = mqtt.connect()) != 0)
  {
    Serial.println();
    Serial.print("MQTT connection failed: ");
    Serial.println(mqtt.connectErrorString(ret));
    mqtt.disconnect();
    delay(5000);
    Serial.print("Retrying...");
  }
  Serial.println();
  Serial.println("Adafruit IO CONNECTED!");
}
// ================================
// LOOP
// ================================
void loop()
{
  // Connect MQTT
  MQTT_connect();
  // -------------------------------
  // READ MQ135
```

```
// ------------------------------
int gasValue = analogRead(MQ135_PIN);
Serial.println();
Serial.println("============================");
Serial.print("MQ135 Value: ");
Serial.println(gasValue);
// ------------------------------
// AIR QUALITY
// ------------------------------
if (gasValue < 400)
{
 digitalWrite(GREEN_LED, HIGH);
 digitalWrite(RED_LED, LOW);
 Serial.println("Air Quality: GOOD");
 // Publish gas value
 if (mq135Feed.publish(gasValue))
 {
  Serial.println("MQ135 data sent successfully!");
 }
 else
 {
  Serial.println("ERROR: MQ135 data NOT sent!");
 }
 // Publish air quality
 if (airQualityFeed.publish("GOOD"))
 {
  Serial.println("Air quality sent successfully!");
 }
```

```
  Else
{
    Serial.println("ERROR: Air quality NOT sent!");
  }
 }
 else
 {
  digitalWrite(GREEN_LED, LOW);
  digitalWrite(RED_LED, HIGH);

  Serial.println("Air Quality: POOR");

  // Publish gas value
  if (mq135Feed.publish(gasValue))
  {
   Serial.println("MQ135 data sent successfully!");
  }
  else
  {
   Serial.println("ERROR: MQ135 data NOT sent!");
  }

  // Publish air quality
  if (airQualityFeed.publish("POOR"))
  {
   Serial.println("Air quality sent successfully!");
  }
  else
  {
```

```
    Serial.println("ERROR: Air quality NOT sent!");
   }
  }
  Serial.println("==============================");
  delay(5000);
}
```

9. System Working :



When the system is powered ON, the ESP8266 initializes the MQ135 sensor and LED outputs. The ESP8266 then connects to the available Wi-Fi network.

After establishing the Wi-Fi connection, it connects to the Adafruit IO cloud platform.

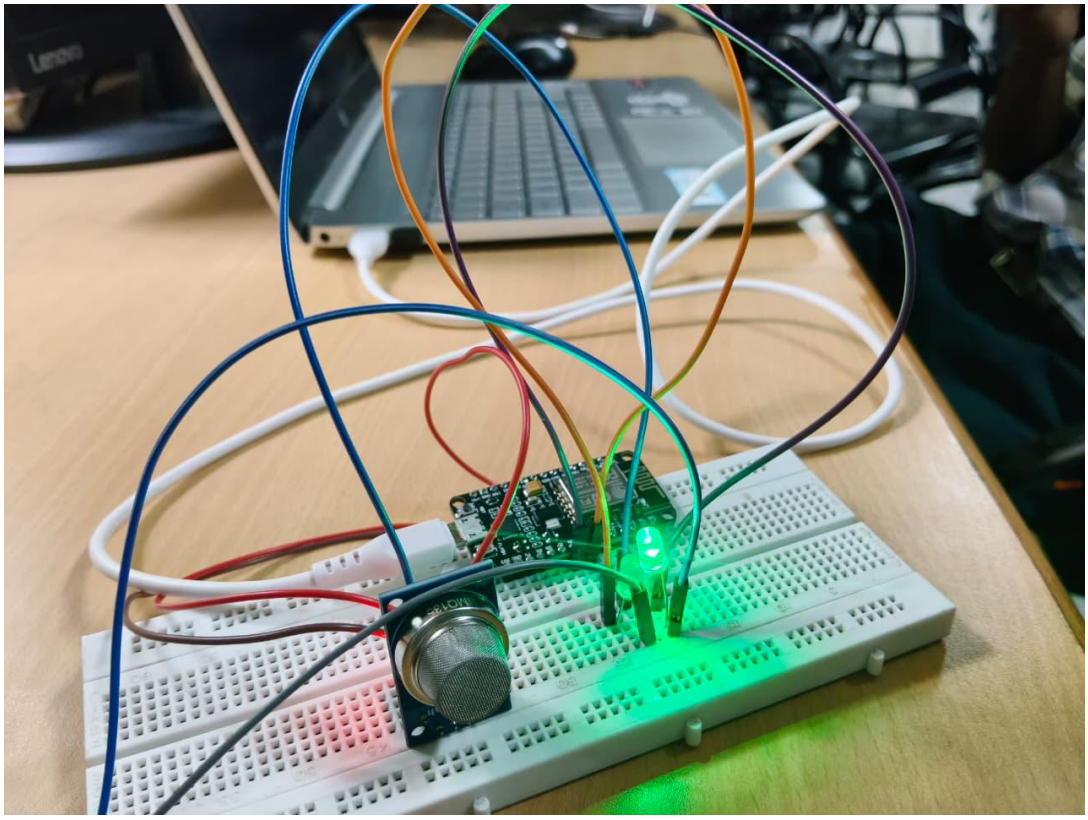
The MQ135 sensor detects gases present in the surrounding air. The analog output of the sensor is read by the ESP8266 through the A0 pin.

10. BILL OF MATERIAL:

BILL OF MATERIAL

S.No.	Component		Specification / Description	Quantity
1.		**ESP8266 NodeMCU**	ESP8266 Wi-Fi Development Board	1
2.		**MQ135 Gas Sensor**	Gas Sensing Module (Analog Output)	1
3.		**Green LED**	5mm LED (Green)	1
4.		**Red LED**	5mm LED (Red)	1
5.		**Resistor**	220 Ω Resistor (for LEDs)	2
6.		**Breadboard**	Solderless Breadboard	1
7.		**Jumper Wires**	Male to Male Jumper Wires	As required
8.		**USB Cable**	Micro USB Cable for Power & Programming	1
9.		**Wi-Fi Connection**	Internet Connection for ESP8266	1
10.		**Adafruit IO Cloud**	Cloud Platform for Data Monitoring	1

11. Prototype Implementation:



The prototype was assembled using an ESP8266 NodeMCU, MQ135 gas sensor and two LEDs.

The MQ135 sensor was connected to the analog input of the ESP8266. The two LEDs were connected to digital pins D5 and D6 through current-limiting resistors.

The ESP8266 was programmed using Arduino IDE. After programming, the controller was connected to a Wi-Fi network.

The prototype was successfully tested by monitoring the sensor values through the Serial Monitor and Adafruit IO cloud dashboard.

12. Results and Performance Analysis:

The developed Air Pollution Monitoring System was successfully implemented using an MQ135 gas sensor, ESP8266 NodeMCU, two LEDs, and Adafruit IO cloud. The system was tested under different air conditions to verify its sensing, indication, and cloud-monitoring capabilities.

The MQ135 sensor continuously detects changes in the surrounding air and provides an analog output to the ESP8266. The ESP8266 processes the sensor reading and compares it with a predefined threshold value. Based on the sensor value, the system identifies the air condition as GOOD or POOR.

When the gas sensor value is below the threshold, the green LED turns ON, indicating good air quality. When the sensor value exceeds the threshold, the red LED turns ON, indicating poor or polluted air conditions.

The ESP8266 was successfully connected to the Wi-Fi network and transmitted the sensor data to the Adafruit IO cloud platform. The MQ135 sensor readings were stored in the mq135 feed, while the air-quality status was stored in the air-quality feed. The data could be monitored remotely through the Adafruit IO dashboard.

During testing, sensor readings such as 52, 63, and 64 were observed under normal conditions, and the system displayed the air-quality status as

GOOD. The successful transmission messages in the Serial Monitor confirmed that the sensor data and air-quality status were being sent to the cloud.

Performance Analysis

The prototype performed successfully in the following areas:

- Gas Detection: The MQ135 successfully detected changes in the surrounding air.
- Data Processing: The ESP8266 successfully read and processed the sensor values.
- LED Indication: The green and red LEDs provided immediate visual indication of air-quality status.
- Wi-Fi Communication: The ESP8266 successfully connected to the Wi-Fi network.
- Cloud Monitoring: Sensor readings were successfully transmitted to Adafruit IO.
- Data Storage: MQ135 readings and air-quality status were successfully stored in separate cloud feeds.
- Remote Monitoring: The air-quality information could be monitored remotely using the Adafruit IO dashboard.
- System Response: The system continuously updated the sensor readings at regular intervals.

Overall, the developed prototype successfully demonstrates the basic concept of an IoT-based air-pollution monitoring system. It provides both local indication through LEDs and remote monitoring through the cloud.